

Plan Overview — ClinicalTrials.gov Query-to-Visualization Agent

The short version: 15 pages, about 15 minutes. Each page covers one idea: a diagram plus the few points that matter. The full plan (`PLAN.pdf`) has every number, schema and rule behind these pages. **Date:** 2026-09-28 · **Time box:** ~24h · **Stack:** Python 3.12 · FastAPI · Pydantic v2 · OpenAI + OpenRouter

Colors used throughout: LLM deterministic code ClinicalTrials.gov API input / output error outcome

Table of Contents

V0 · One-page summary	V8 · Anatomy of a deep citation
V1 · The system in one picture	V9 · The verifier: trust, but recount
V2 · One request, end to end	V10 · What the response looks like
V3 · The revise loop (every planning outcome)	V11 · How the code is organized
V4 · What the planner is allowed to say (QueryPlan)	V12 · The 24-hour build plan
V5 · From question shape to chart	V13 · What questions it answers
V6 · The eight chart types	V14 · Four decisions for you
V7 · Where every trial goes (the data funnel)	

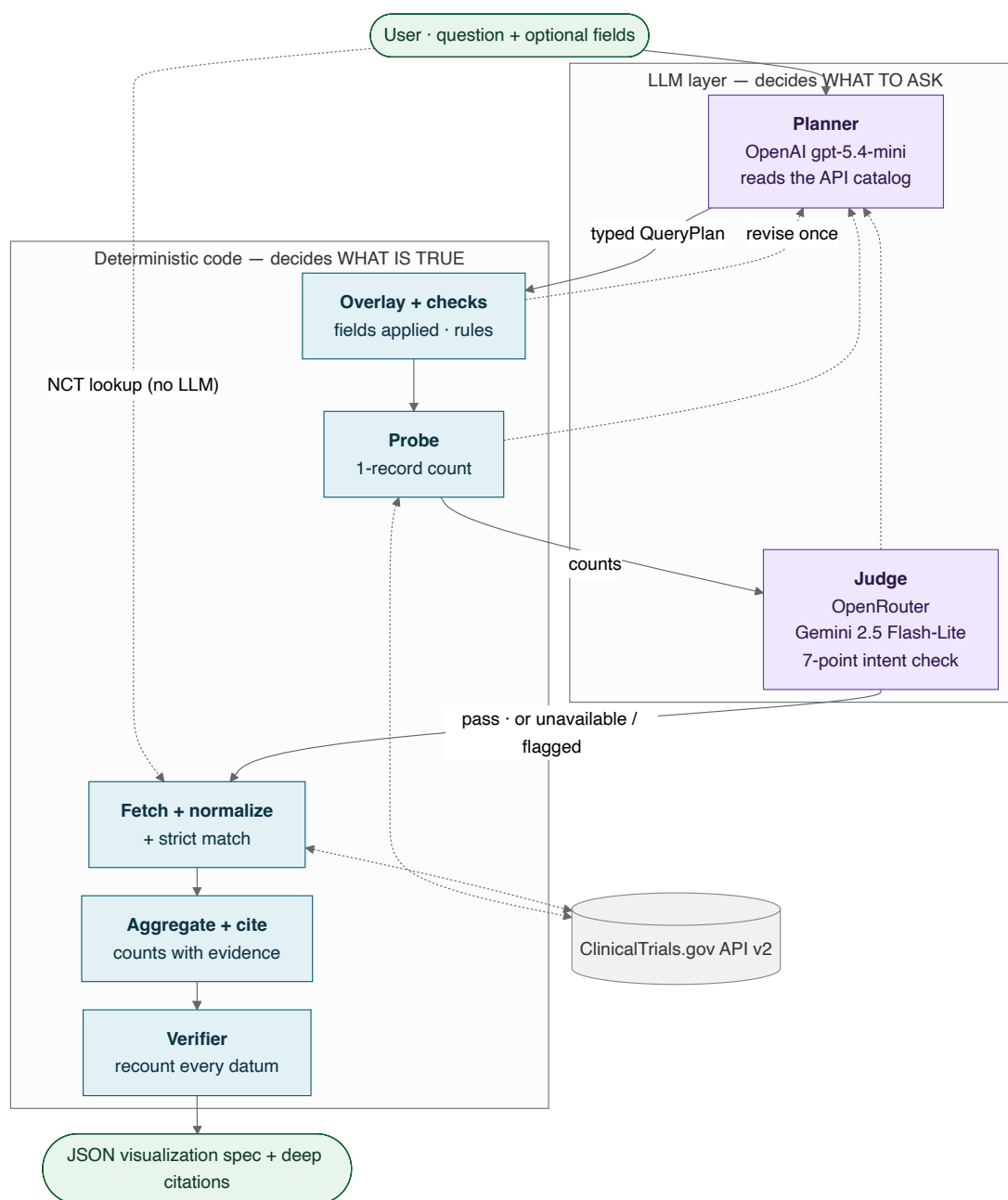
The one rule behind every design choice: *LLMs decide **what to ask**; code decides **what is true**.*

V0 · One-page summary

What	A FastAPI backend that turns a natural-language clinical-trial question (plus optional fields) into a JSON visualization spec backed by ClinicalTrials.gov, with a citation for every trial behind every datum .
How	OpenAI plans , choosing from menus only. Code checks the plan and probes real counts. OpenRouter judges the plan. Then code fetches, counts, cites and verifies .
Why it's trustworthy	No number, category label or excerpt is written by an LLM. An independent verifier recounts every datum and refuses to answer if anything is off.
Coverage	8 chart types (bar, grouped bar, time series, histogram, scatter, network, table, metric) and 19 query types (18 core + 1 stretch), covering every class in the assignment's appendix. API totals were measured live for 17 of them.
Schedule	About 24 h. A working end-to-end slice by hour 6.5, deep citations by hour 13.5, and 2.5 h reserved. The deliverables phase is protected.
What I need from you	Four quick decisions (page V14).

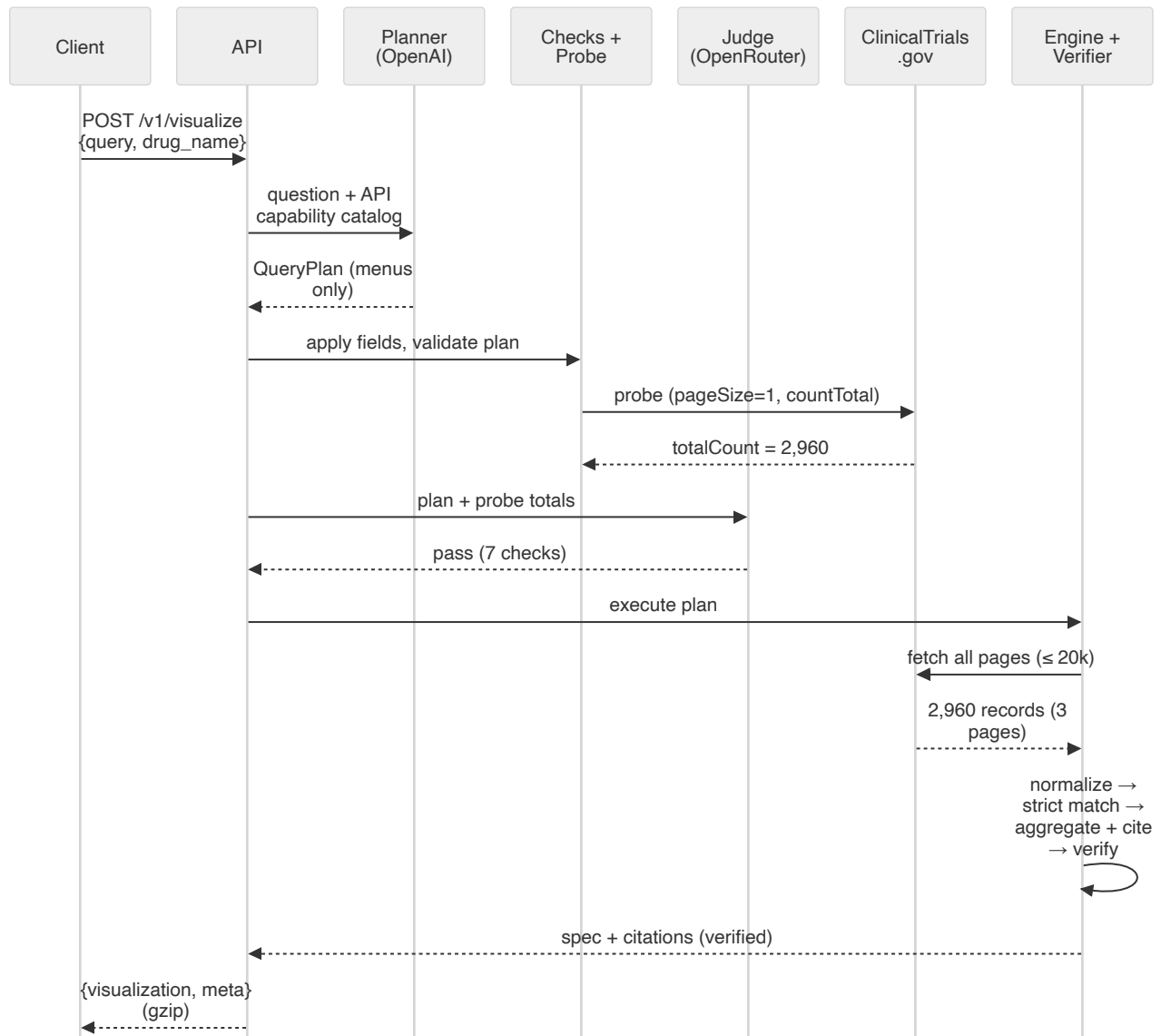
The one rule behind every design choice: *LLMs decide **what to ask**; code decides **what is true**.*

V1 · The system in one picture



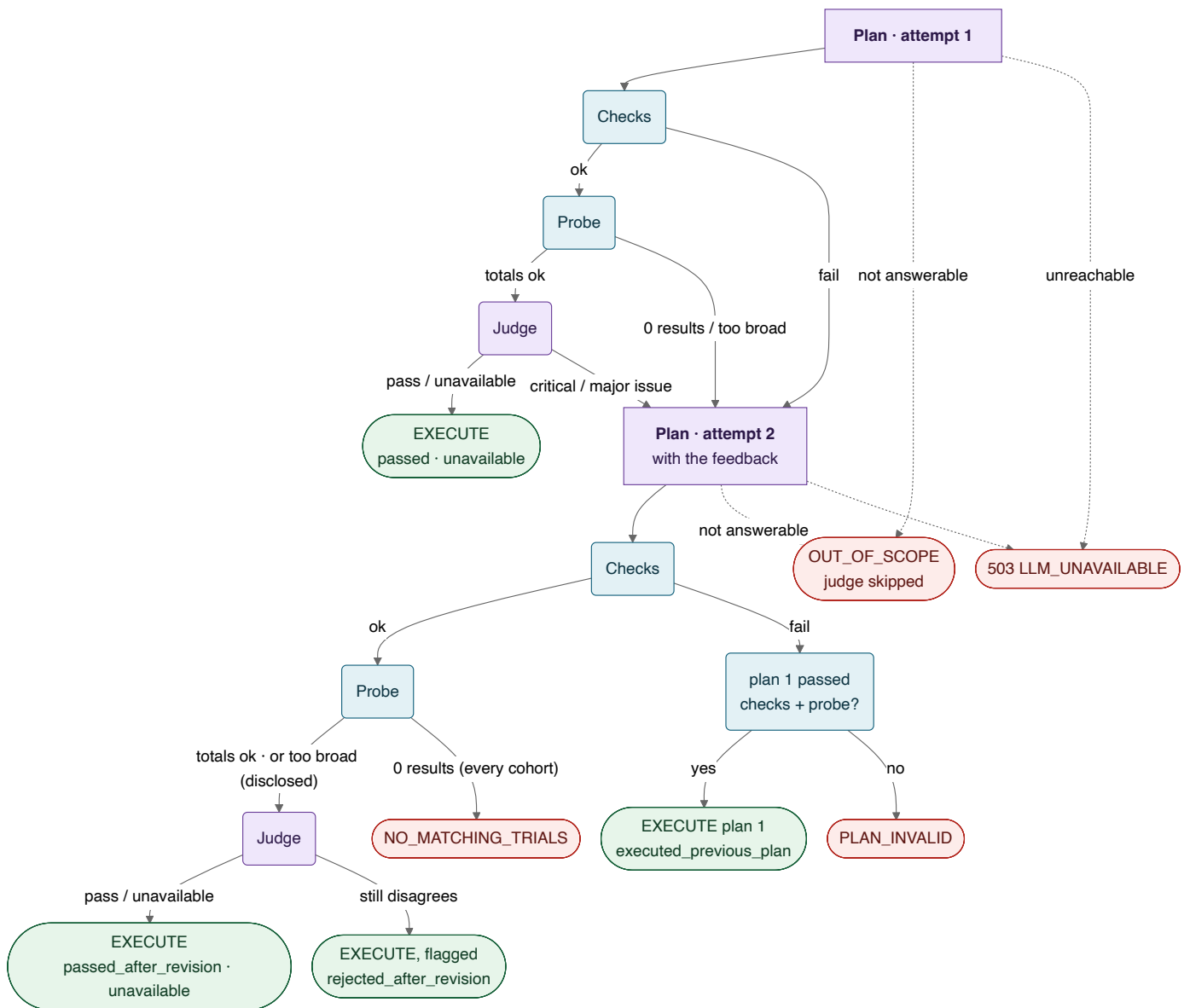
- **The LLMs only produce a typed plan:** menu choices plus entity names such as "pembrolizumab". They never write URLs, numbers or excerpts.
- **The plan is tested three ways before any real data is fetched:** code checks, a live count probe, and a judge from a *different* model family.
- **Any of the three checks can send the plan back, but only once** (dotted arrows), so latency is bounded: typically 3–9 s.
- **Everything after the judge is plain, testable Python.**

V2 · One request, end to end



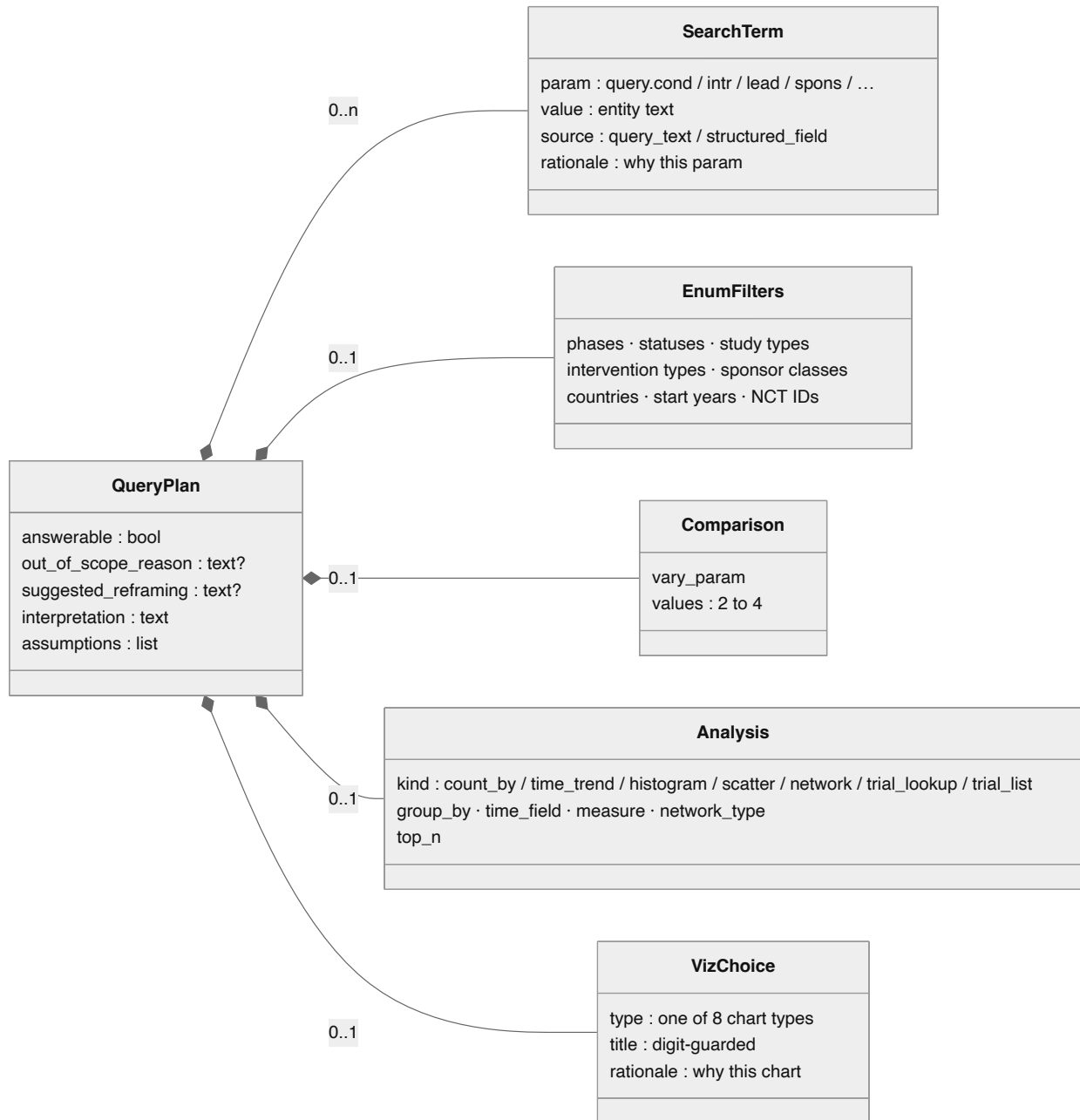
- The example is the assignment's own request: "How has the number of trials for this drug changed over time?" with `drug_name = Pembrolizumab`.
- **Typical timings:** planner 1–3 s · probe < 0.5 s · judge 0.5–1.5 s · fetch ~3–4 s for 2,960 records · verify < 0.5 s.
- **LLM cost is about \$0.01 per request.**

V3 · The revise loop (every planning outcome)



- **At most 2 planner calls + 2 judge calls.** The loop can't spin.
- **Every ending is named.** Executed answers carry `meta.validation.judge.status` (passed · passed_after_revision · rejected_after_revision · executed_previous_plan · unavailable). The rest return `ok:false` with an `error.code` (OUT_OF_SCOPE · PLAN_INVALID · NO_MATCHING_TRIALS · LLM_UNAVAILABLE).
- **The judge can't block forever:** if it still disagrees after the revision, the answer ships *flagged*, with its objections attached.
- **If the judge is unreachable, the plan still runs (fail open), flagged `unavailable`.** Correctness is protected by the code checks, not by the judge.
- **Simple NCT-ID lookups bypass this loop entirely** (`judge.status = skipped`).

V4 · What the planner is allowed to say (QueryPlan)



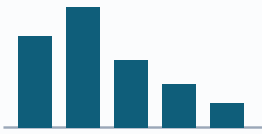
- **Every search, filter, analysis and chart field is a menu choice or an entity name.** OpenAI's strict JSON-schema mode enforces this, then the §7.4 code checks. The only free text (title, interpretation, assumptions, rationales) is digit-guarded or shown for transparency, and is never used to query.
- **The rationale fields are the planner's reasoning** over the API catalog ("what does each call provide?"). The judge reads them.
- **Structured request fields (e.g. drug_name) are written in by code** afterwards, never copied by the LLM.

V5 · From question shape to chart



- **The planner picks both the kind and the chart; code enforces that they're compatible.** For example, a `time_trend` can't become a network.
- **After the data arrives, deterministic *shape guards* adjust degenerate cases:**
 - 1 bucket → metric
 - < 3 years → bar chart
 - too many nodes → pruning
- **Simple NCT-ID lookups** (the IDs plus ≤ 6 words, with no 'by / vs / trend...') take a no-LLM fast path. Other NCT questions go to the planner with the IDs pre-filled.

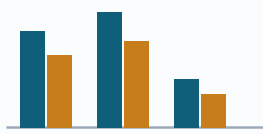
V6 · The eight chart types



bar_chart

x: category · y: trial_count

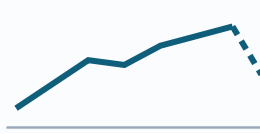
Breast cancer trials by phase



grouped_bar_chart

x · y · color = cohort

Phases: pembrolizumab vs nivolumab



time_series

x: year · y: trial_count · partial
year dashed

Pembrolizumab trials per year



histogram

bins: bin_start / bin_end · y:
trial_count

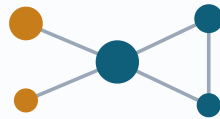
Enrollment sizes, Phase 2 psoriasis



scatter_plot

one point = one trial · x · y

Enrollment vs duration, Crohn's P3



network_graph

nodes · edges · weight =
shared trials

Sponsors ↔ drugs, glioblastoma

table

columns[] · one row per item

Recruiting Phase 3 trials for X

COMPLETED

NCT04368728 · status

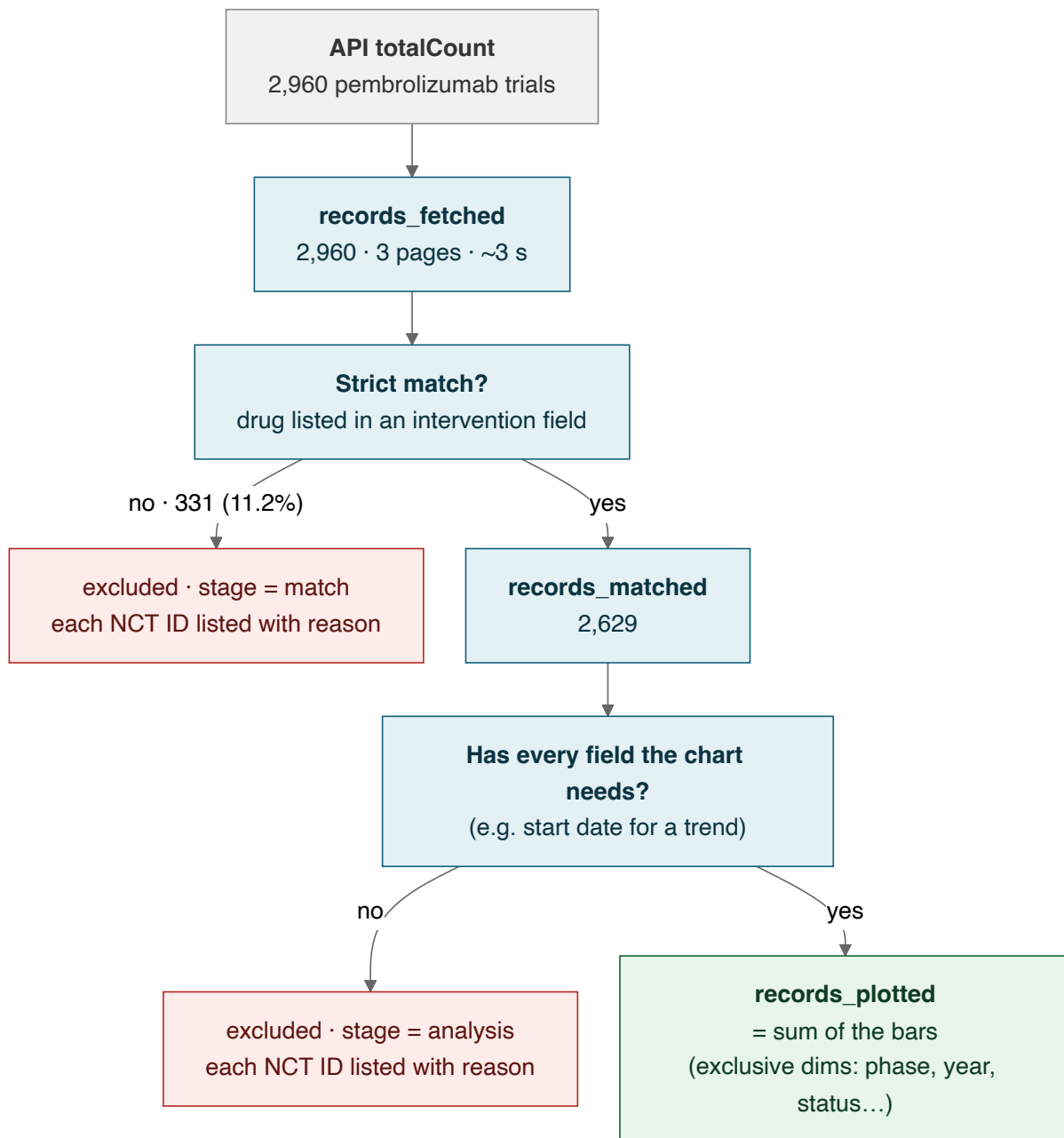
metric

label · value · unit (data is always an array)

Status of NCT04368728?

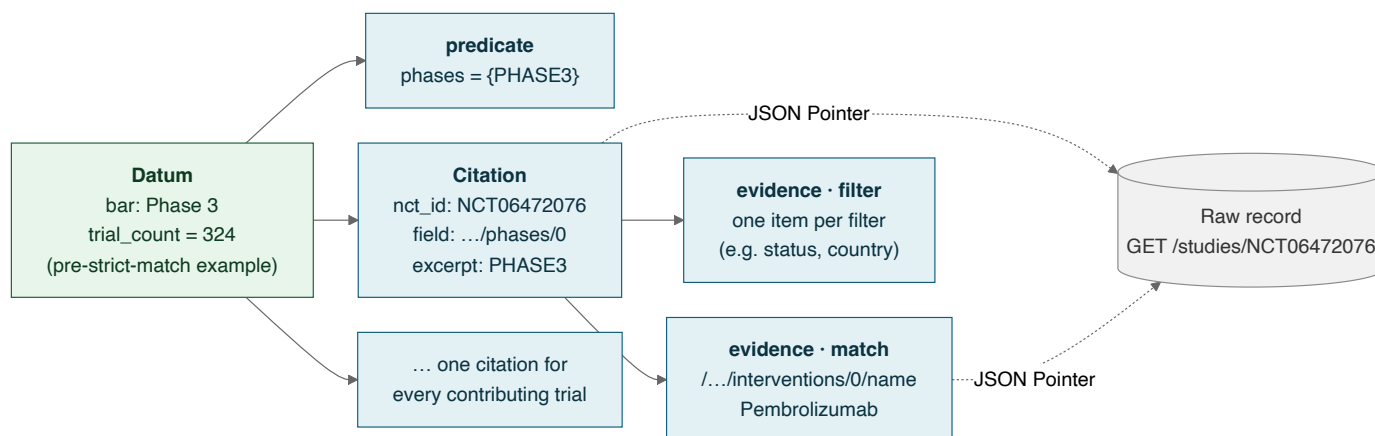
- **Every type has the same five keys:** `type`, `title`, `encoding`, `data`, `options`.
- `encoding` names exactly which data field drives which visual channel, so a frontend never guesses.
- **Data is already aggregated.** The frontend only draws; it never recounts.

V7 · Where every trial goes (the data funnel)



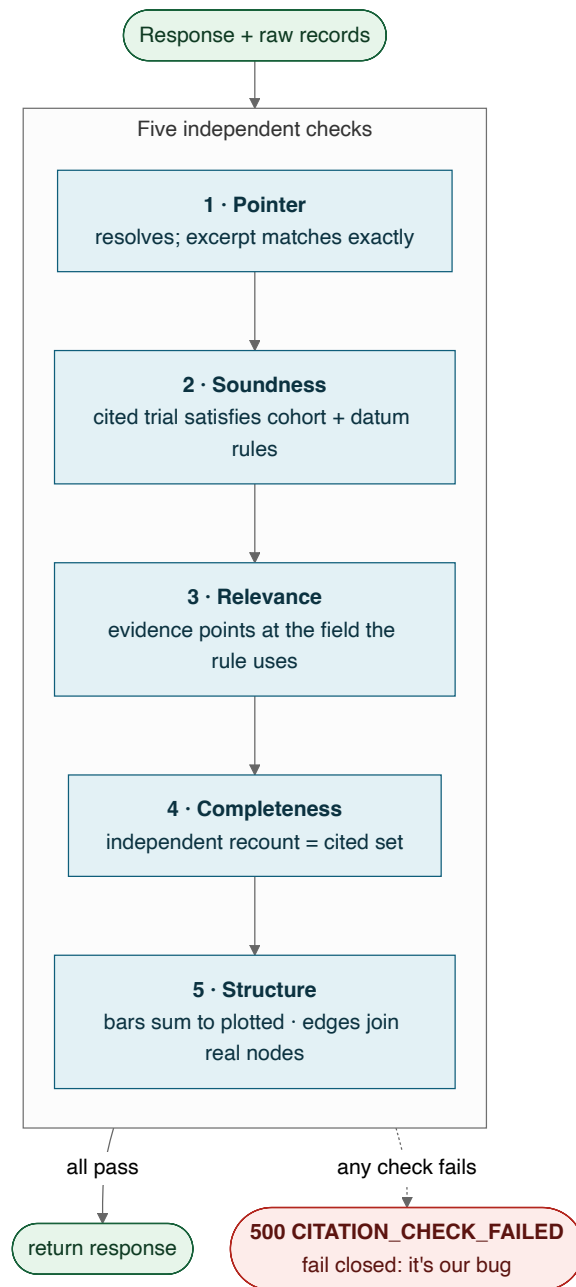
- **The API's text search is fuzzy:** it also matches trials that only *mention* the drug in a description. 331 of 2,960 pembrolizumab hits don't actually list it **VERIFIED**.
- **Nothing disappears silently.** Every excluded trial is listed with its stage and reason in `meta.data_coverage`.
- **Over 20,000 matches?** The probe asks the planner to narrow the question first. Otherwise the most recent 20,000 are analyzed, and the response says so.

V8 · Anatomy of a deep citation



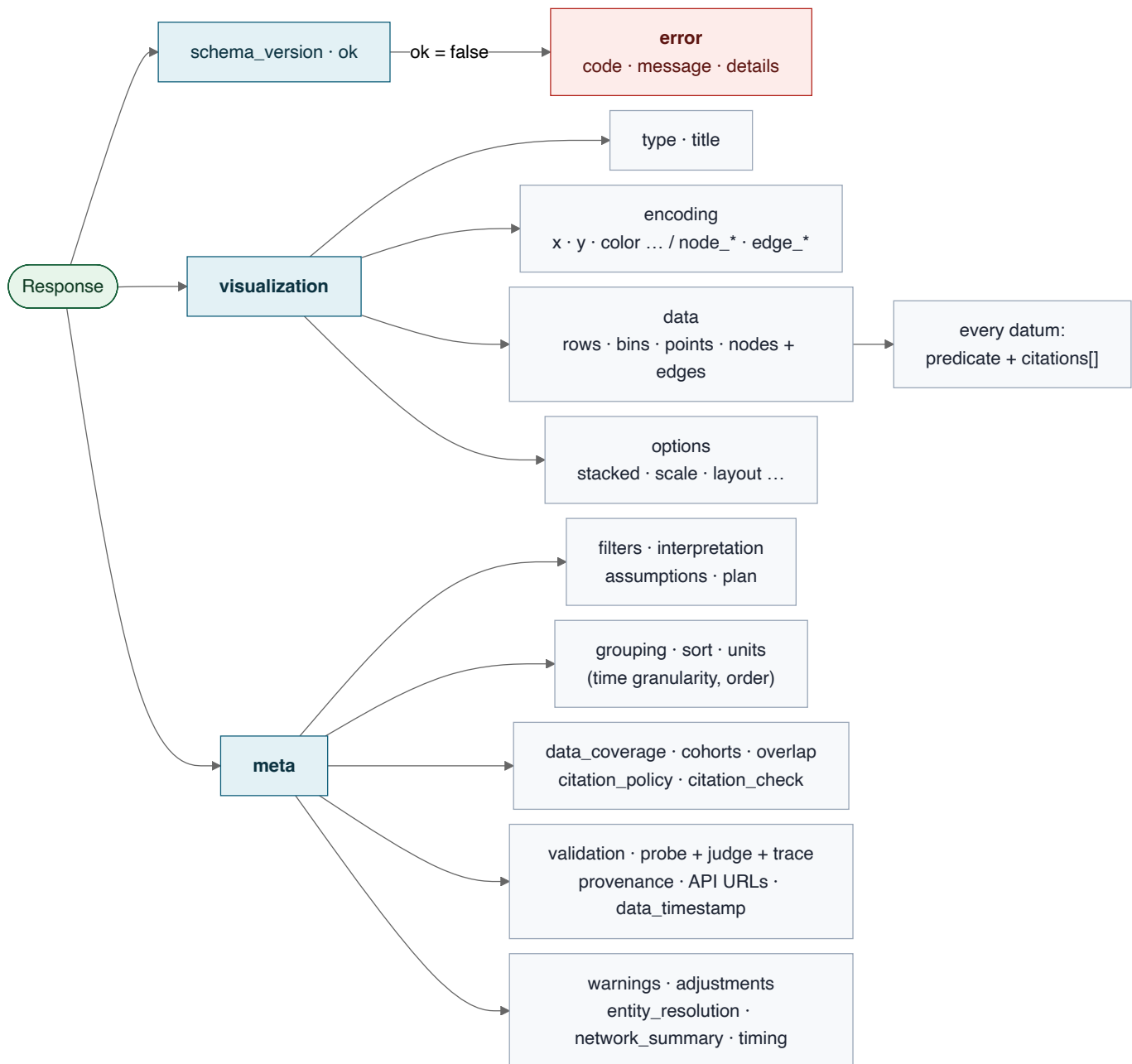
- Each citation answers two questions: **why is this trial in this bar** (`field` / `excerpt`), and **why is it in the chart at all** (`match` evidence).
- **Excerpts are exact values straight from the API**, one array element at a time, never paraphrased.
- `trial_count` equals the number of citations, so the bar height is its citation list.
- **Anyone can check a citation** with any JSON Pointer library, or with `python -m ctviz.verify`.

V9 · The verifier: trust, but recount



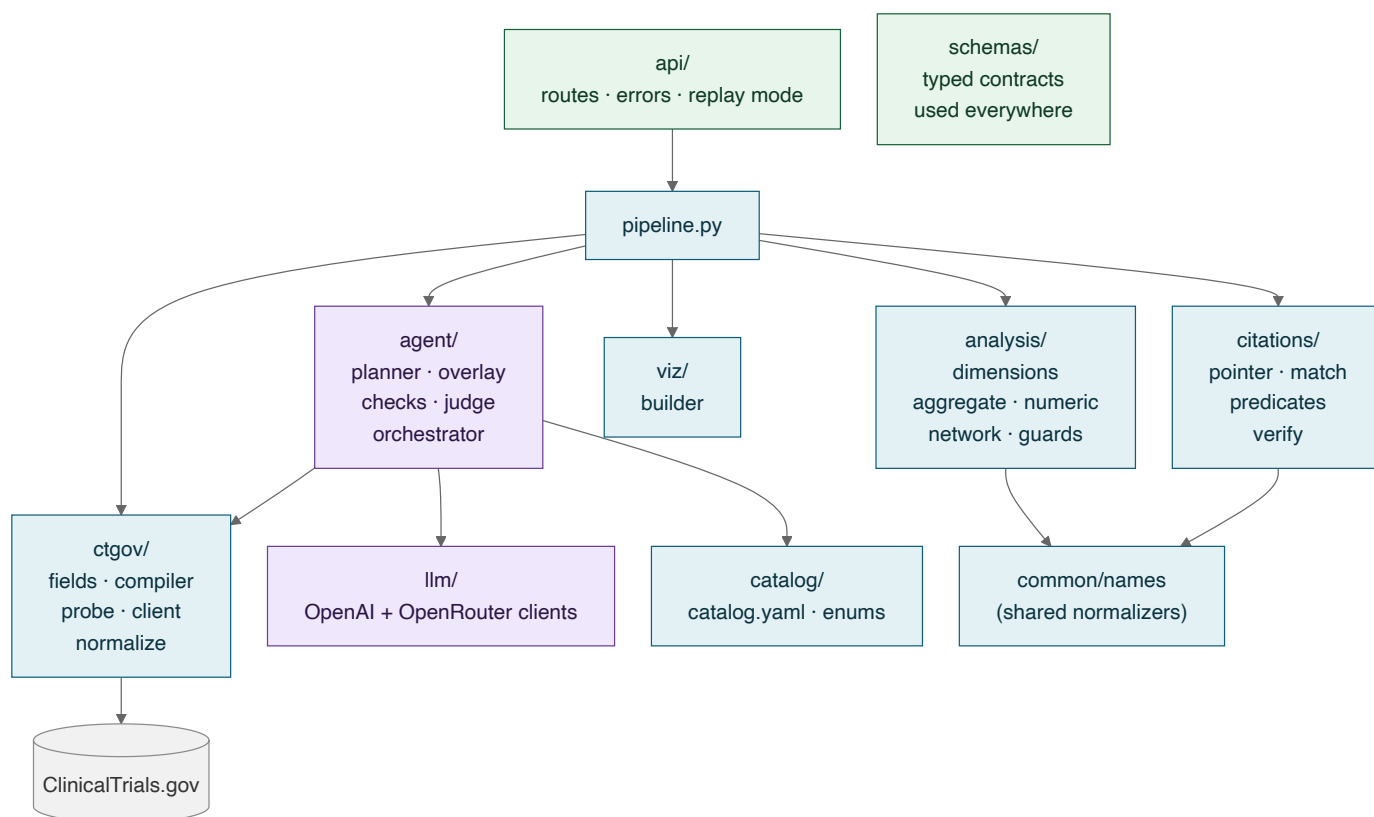
- **The verifier never imports the counting code.** It re-derives every bar from the raw records using each datum's predicate, so a bucketing bug can't hide.
- **It catches the failure the first prototype missed:** a Phase 1 trial filed under the Phase 3 bar.
- **Nine deliberate-corruption tests prove it fails when it should.**

V10 · What the response looks like



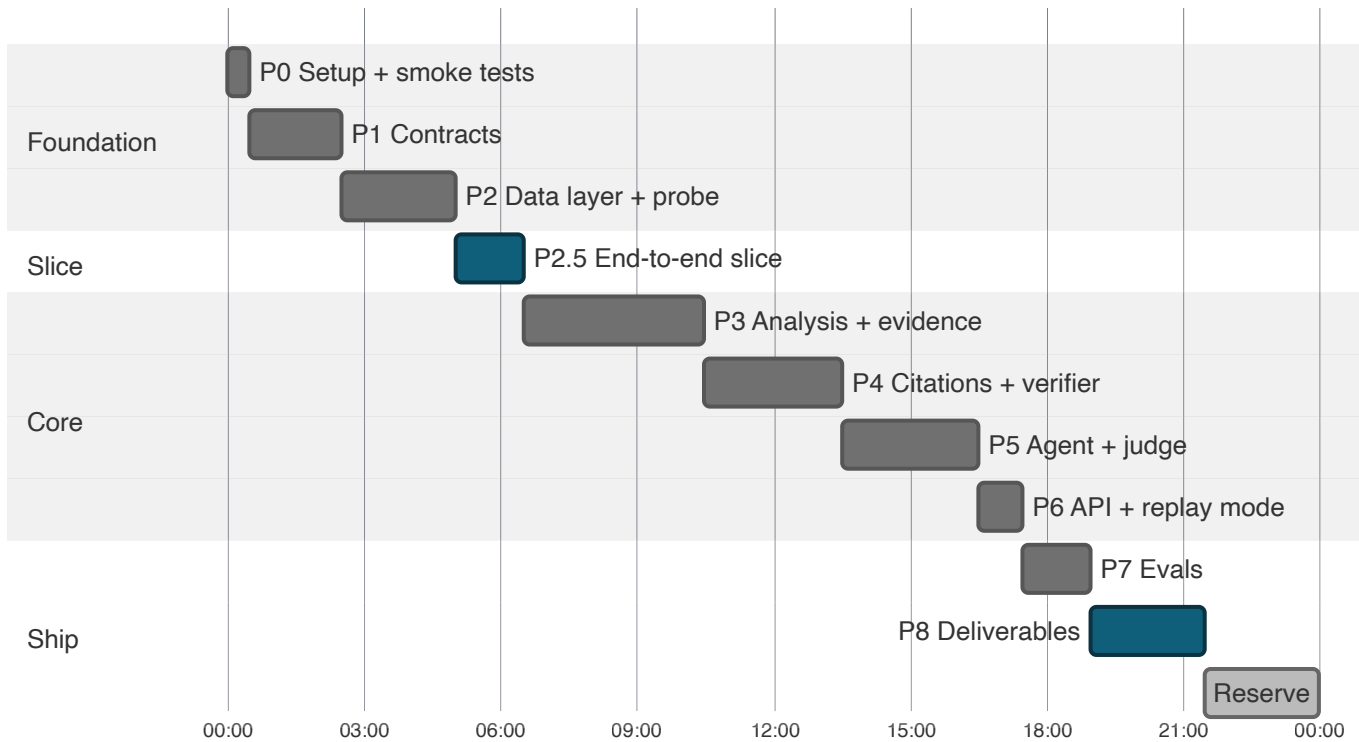
- **One parse path for any frontend:** check `ok`, then render `visualization` using `encoding`.
- **meta explains the answer:** what was asked, which trials were counted or dropped and why, what the judge said, and which exact API calls produced the data.

V11 · How the code is organized



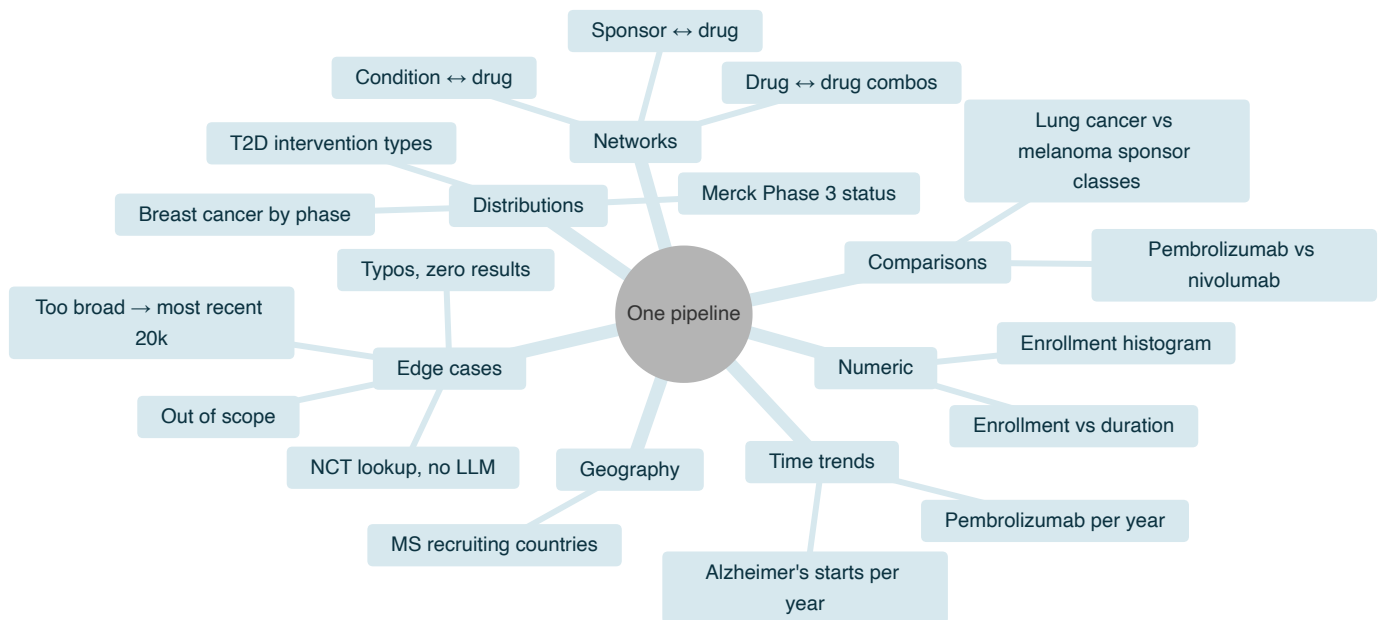
- **Small typed modules** (~200–400 lines each), with one-way dependencies.
- `citations/` **never imports** `analysis/`. That independence is what makes the verifier meaningful.
- **LLM code is isolated in** `agent/` + `llm/`. Everything else is testable offline with recorded API fixtures.

V12 · The 24-hour build plan



- **A real end-to-end answer exists by hour 6.5**, so problems surface early.
- **The 2.5 h reserve covers a short review after each phase, plus slack.**
- **If behind at hour 16, cut in this order:** stretch items → judge eval size → condition↔drug network → scatter → table lists. **P8 (deliverables) is never cut.**
- **Stretch, only after P8:** site and investigator networks, then the demo UI (only if ≥ 2 h remain).

V13 · What questions it answers



- **18 core query types (+1 stretch: the site network) run through the same pipeline**, with no per-question code. API totals for 17 were measured live (full plan §14); the Pfizer condition↔drug total is still an **ASSUMPTION**.
- **Every class in the assignment's appendix is covered**, including the unscoped "Which drugs frequently co-occur in combination studies?".

V14 · Four decisions for you

#	Question	My recommendation
Q1	Exclude trials the API matched only by <i>mentioning</i> the drug or sponsor in passing (11 % for pembrolizumab)?	Yes for drugs and sponsors. Conditions stay lenient until P2 measures their match rate (switch to strict if $\geq 97\%$). Every excluded trial is listed.
Q2	"Merck" matches two unrelated companies (MSD and Merck KGaA). Warn, or auto-split?	Warn and show the matched names , without guessing.
Q3	Build a small HTML demo viewer?	Only if ≥ 2 h remain after the deliverables are done.
Q4	Should a "Phase 2/Phase 3" trial be its own bar (sums to the total, like ClinicalTrials.gov) or count in both?	Its own bar. (Counting in both stays available as stretch S5.)

Answers are needed before P2 / P4, so P0–P1 can start right away. Reply with your answers, or "go with your recommendations".